

FORMATION EMBARQUÉE

TD 02 Cpp

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 02 — C++ : énoncés et corrigés détaillés

Compile avec `g++ -Wall -Wextra -std=c++17` . Chaque corrigé illustre un concept central du cours : templates, polymorphisme, RAII, FSM objet.

Exercice 1 — Tampon circulaire générique `TamponCirculaire<T, N>`

Énoncé. Réécris le ring buffer du TD 01 en classe template, avec tests.

Corrigé

```
#include <cstdint>
#include <cstdint>

template <typename T, std::size_t N>
class TamponCirculaire {
    static_assert(N >= 2, "taille minimale : 2");
    static_assert((N & (N - 1)) == 0, "N doit être une puissance de 2");

public:
    bool put(const T& valeur) {
        if (plein()) return false;
        donnees_[tete_] = valeur;
        tete_ = (tete_ + 1) & MASQUE;
        return true;
    }

    bool get(T& valeur) {
        if (vide()) return false;
        valeur = donnees_[queue_];
        queue_ = (queue_ + 1) & MASQUE;
        return true;
    }

    bool vide() const { return tete_ == queue_; }
    bool plein() const { return ((tete_ + 1) & MASQUE) == queue_; }

    std::size_t taille() const { // nb d'éléments présents
        return (tete_ - queue_) & MASQUE;
    }

private:
    static constexpr std::size_t MASQUE = N - 1;
    T donnees_[N] {}; // stockage STATIQUE : zéro allocation
    volatile std::size_t tete_ = 0;
    volatile std::size_t queue_ = 0;
};
```

Tests (avec `assert` , compilables sur PC) :

```

#include <cassert>

int main() {
    TamponCirculaire<std::uint8_t, 8> tc;          // 8 cases → 7 utilisables
    assert(tc.vide() && !tc.plein());

    for (std::uint8_t i = 0; i < 7; i++) assert(tc.put(i));
    assert(tc.plein());
    assert(!tc.put(99));                          // refus quand plein

    std::uint8_t v;
    for (std::uint8_t i = 0; i < 7; i++) { assert(tc.get(v)); assert(v == i); }
    assert(tc.vide() && !tc.get(v));              // refus quand vide

    TamponCirculaire<float, 4> tf;                // le MÊME code pour un autre type
    tf.put(3.14f);
    return 0;
}

```

Ce que le correcteur regarde : - Les `static_assert` : les erreurs de dimensionnement sont attrapées à la compilation — impossible en C. - `T donnees_[N]` : la taille fait partie du type ; deux tampons de tailles différentes sont deux types distincts, tout est sur la pile ou en statique. - Le template est **instancié à la demande** : `TamponCirculaire<float,4>` et `<uint8_t,8>` génèrent deux codes spécialisés — abstraction sans coût.

Exercice 2 — Interface `IAfficheur` et polymorphisme

Énoncé. Interface abstraite `IAfficheur` (`effacer()` , `texte(x,y,str)`) + deux implémentations (console, « LCD » factice) ; le même code applicatif doit tourner sur les deux.

Corrigé

```
#include <cstdint>
#include <cstdio>
#include <cstring>

class IAfficheur {
public:
    virtual ~IAfficheur() = default;          // OBLIGATOIRE : destructeur virtuel
    virtual void effacer() = 0;               // = 0 : méthode pure → interface
    virtual void texte(std::uint8_t x, std::uint8_t y, const char* s) = 0;
};

// Implémentation 1 : la console du PC (pour développer sans matériel)
class AfficheurConsole : public IAfficheur {
public:
    void effacer() override { std::puts("\n--- effacé ---"); }
    void texte(std::uint8_t x, std::uint8_t y, const char* s) override {
        std::printf("[%2u,%2u] %s\n", x, y, s);
    }
};

// Implémentation 2 : un "LCD" 16x2 simulé en mémoire
class AfficheurLcd16x2 : public IAfficheur {
public:
    void effacer() override { std::memset(ecran_, ' ', sizeof ekran_); }
    void texte(std::uint8_t x, std::uint8_t y, const char* s) override {
        if (y >= 2) return;                  // on borne TOUJOURS
        for (std::uint8_t i = 0; s[i] != '\0' && (x + i) < 16; i++)
            ekran_[y][x + i] = s[i];
    }
    void dump() const {                      // aide au debug/test
        std::printf("|%.16s|\n|%.16s|\n", ekran_[0], ekran_[1]);
    }
private:
    char ekran_[2][16] {};
};

// ---- Code applicatif : il ne connaît QUE l'interface ----
void afficher_mesure(IAfficheur& aff, float temperature) {
    char ligne[17];
    std::snprintf(ligne, sizeof ligne, "T=%.1f C", temperature);
    aff.effacer();
    aff.texte(0, 0, "Station meteo");
    aff.texte(0, 1, ligne);
}

int main() {
    AfficheurConsole console;
    AfficheurLcd16x2 lcd;
    afficher_mesure(console, 21.5f);         // même fonction...
    afficher_mesure(lcd, 21.5f);             // ...deux matériels
    lcd.dump();
}
```

Leçon centrale : `afficher_mesure` est écrite **une fois** et testée sur PC avec `AfficheurConsole` — le vrai driver LCD n'arrive qu'à la fin. C'est la technique qui rend un firmware **testable sans matériel** ; en entreprise elle s'appelle « injection de dépendances » et l'implémentation console un « mock ».

Exercice 3 — Classe RAI `ChipSelect`

Énoncé. Une classe qui abaisse la broche CS dans le constructeur et la relâche dans le destructeur.

Corrigé

```
class ChipSelect {
public:
    explicit ChipSelect(std::uint8_t broche) : broche_(broche) {
        digitalWrite(broche_, LOW);          // CS actif à l'état bas : on sélectionne
    }
    ~ChipSelect() {
        digitalWrite(broche_, HIGH);         // désélection GARANTIE

        // Un objet RAI ne doit être ni copié ni déplacé :
        // deux copies relâcheraient CS deux fois.
        ChipSelect(const ChipSelect&) = delete;
        ChipSelect& operator=(const ChipSelect&) = delete;

private:
    std::uint8_t broche_;
};

// Utilisation :
std::uint8_t lire_registre_spi(std::uint8_t adresse) {
    ChipSelect cs(PIN_CS);                  // CS descend ici
    SPI.transfer(adresse | 0x80);
    std::uint8_t v = SPI.transfer(0x00);
    if (v == 0xFF)                          // ← sortie anticipée : CS remonte QUAND MÊME
        return 0;
    return v;                               // ← CS remonte ici dans le cas normal
}
```

Pourquoi c'est mieux que deux appels manuels : la sortie anticipée (`return` , `break` , exception) fait remonter CS **automatiquement**. La version manuelle oublie toujours un chemin de sortie un jour ou l'autre — et un CS resté bas bloque tout le bus SPI. Le `= delete` de la copie fait partie de la réponse attendue.

Exercice 4 — FSM feu tricolore en classe

Énoncé. Transformer la FSM du TD 01 en classe avec `tick(maintenant_ms)`.

Corrigé

```
#include <cstdint>

class FeuTricolore {
public:
    enum class Etat : std::uint8_t { Vert, Orange, Rouge };

    explicit FeuTricolore(std::uint32_t maintenant)
        : etat_(Etat::Rouge), t_entree_(maintenant) {}

    void appuiPieton() { demande_pieton_ = true; }
    Etat etat() const { return etat_; }

    void tick(std::uint32_t maintenant) {
        const std::uint32_t ecoule = maintenant - t_entree_;
        switch (etat_) {
            case Etat::Vert:
                if (ecoule >= DUREE_VERT ||
                    (demande_pieton_ && ecoule >= VERT_MINI))
                    transition(Etat::Orange, maintenant);
                break;
            case Etat::Orange:
                if (ecoule >= DUREE_ORANGE) {
                    demande_pieton_ = false;           // demande servie
                    transition(Etat::Rouge, maintenant);
                }
                break;
            case Etat::Rouge:
                if (ecoule >= DUREE_ROUGE)
                    transition(Etat::Vert, maintenant);
                break;
        }
    }

private:
    void transition(Etat suivant, std::uint32_t maintenant) {
        etat_ = suivant;
        t_entree_ = maintenant;           // UN SEUL endroit où l'on change d'état
    }

    static constexpr std::uint32_t DUREE_VERT      = 5000;
    static constexpr std::uint32_t DUREE_ORANGE    = 1000;
    static constexpr std::uint32_t DUREE_ROUGE     = 5000;
    static constexpr std::uint32_t VERT_MINI       = 1000;

    Etat          etat_;
    std::uint32_t t_entree_;
    bool          demande_pieton_ = false;
};
```

Gains par rapport à la version C : - L'état et son horodatage sont **encapsulés** : personne ne peut mettre la FSM dans un état incohérent de l'extérieur (en C, n'importe qui pouvait écrire `f.etat = VERT;` sans

réinitialiser le chrono). - `enum class` : `Etat::Vert` ne se convertit pas silencieusement en entier et ne pollue pas l'espace de noms. - Le constructeur garantit un départ valide ; `transition()` centralise le changement d'état — un seul endroit à instrumenter pour tracer/déboguer. - On peut instancier **plusieurs feux indépendants** (`FeuTricolore nord, sud;`) — la version C à variables globales ne le permettait pas.

Exercice 5 (complément) — Lecture critique

Énoncé. Ce code compile. Donne trois critiques d'un point de vue « C++ embarqué » et propose les corrections.

```
class Journal {
public:
    Journal() { buffer = new char[4096]; }
    ~Journal() { }
    void log(std::string msg) { lignes.push_back(msg); }
private:
    char* buffer;
    std::vector<std::string> lignes;
};
```

Corrigé

1. **Fuite mémoire** : `new[]` dans le constructeur, jamais de `delete[]` dans le destructeur. Et si on ajoute le `delete[]`, la classe devient dangereuse à copier (double libération) → il faudrait aussi gérer ou supprimer copie/affectation (« règle des trois »). Correction embarquée : `char buffer[4096];` en membre (statique, pas de gestion) ou `std::unique_ptr<char[]>` sur gros système.
2. **`std::string` passé par valeur** : copie (et allocation) à chaque appel. Correction : `const std::string&`, ou mieux en embarqué `const char*` / `std::string_view`.
3. **`std::vector<std::string>` qui grossit sans borne** : allocations dynamiques répétées, fragmentation, et croissance illimitée sur un système qui tourne des mois → un jour, plus de RAM. Correction : journal **circulaire** de taille fixe (notre `TamponCirculaire` d'entrées de taille bornée), politique d'écrasement des plus anciens documentée.

Auto-évaluation avant la suite

Sans notes : expliquer RAI avec un exemple matériel ; justifier destructeur virtuel + `= delete` de la copie ; différence polymorphisme dynamique (vtable) / statique (template) et leurs coûts ; citer 3 éléments de la STL utilisables sur micro et 3 à éviter, avec la raison.